

COMPARING SOFTWARE PROCESS MODELS

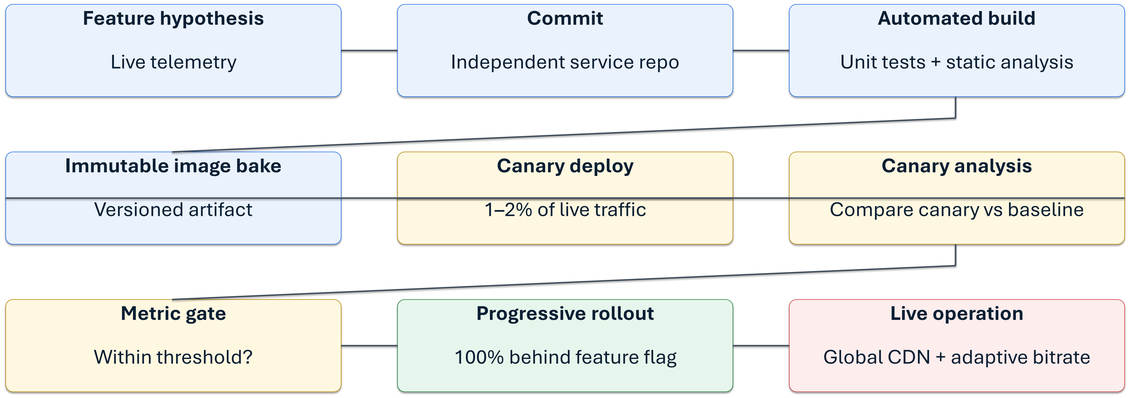
IN CLOUD-NATIVE AND LIVE-SERVICE ARCHITECTURES

An Empirical Analysis of Netflix and Riot Games

Truong Phat Tu • Concordia University Chicago • Software Engineering (BLA 02)
Faculty Sponsor: Dr. Visvasuresh (Victor) Govindaswamy

CORE FINDING **Architecture constrains process cadence: independently deployable services enable continuous release; coupled client–server systems favor synchronized release trains. There is no universal best process.**

01 NETFLIX | Continuous DevOps — DECOUPLED MICROSERVICES

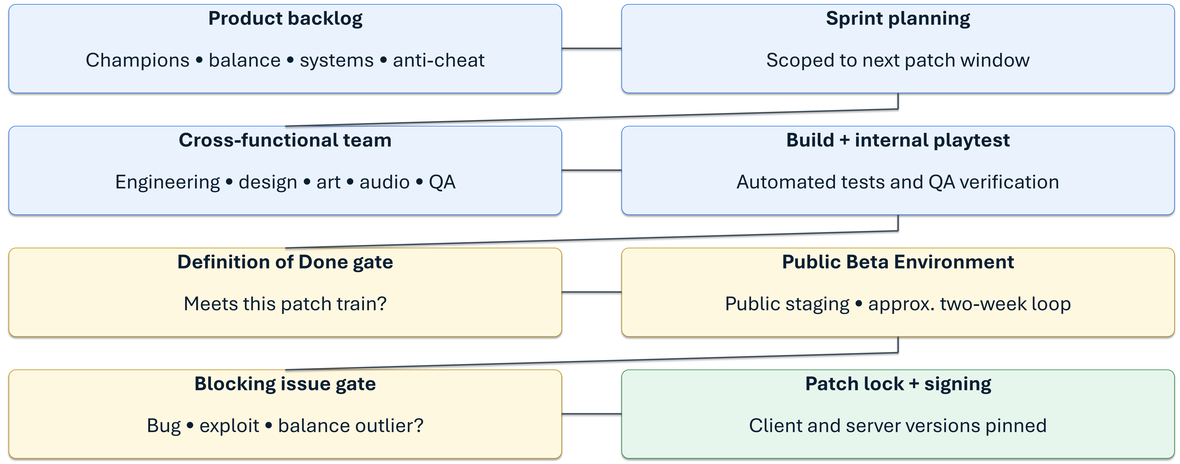


VALIDATION OUTCOMES

PASS → Progressive rollout
FAIL → Automatic rollback; traffic returns to baseline
Production resilience checks
Chaos Engineering fault injection

Continuous telemetry + A/B experiments feed the next hypothesis — potentially the same day.

02 RIOT GAMES | Hybrid Agile — CLIENT–SERVER PARITY



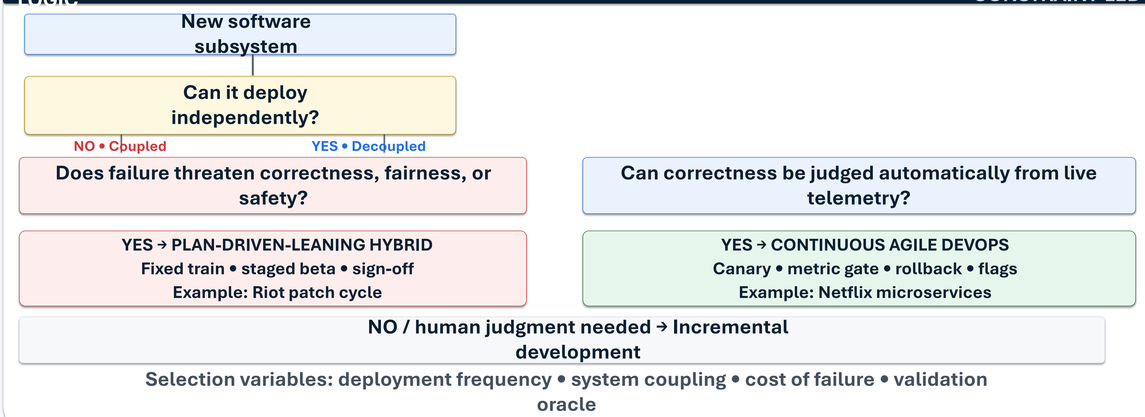
RELEASE & LIVE SERVICE

No → defer scope to next patch • Yes → synchronized regional rollout
Critical regression → emergency hotfix
Live telemetry + community feedback → next bi-weekly increment

03 SIDE-BY-SIDE COMPARISON — ARCHITECTURE → CADENCE

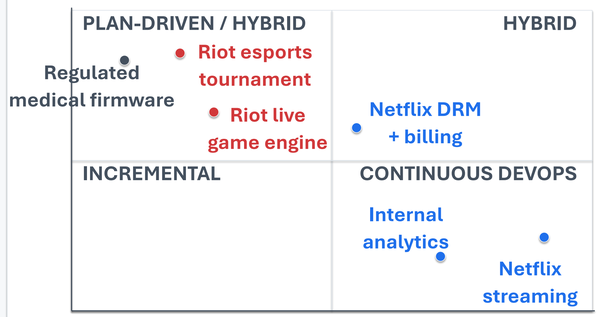
DIMENSION	NETFLIX	RIOT GAMES
Release unit	Independent microservice	Coupled client + server
Cadence	Continuous; can ship same day	Fixed bi-weekly patch train
Primary validation	Automated canary metrics; fault injection	Internal playtest + public beta
Failure response	Metric-gated rollback	Hotfix loop / emergency hotfix

04 PROCESS SELECTION LOGIC — CONSTRAINT-LED



Selection variables: deployment frequency • system coupling • cost of failure • validation oracle

05 DECISION MATRIX — ANALYTICAL



Axes: x = deployment frequency | y = coupling / failure cost
Positions are analytical placements—not measured data.

06 ENGINEERING TAKEAWAYS — SYNTHESIS

01 **Decoupling enables independent shipping.**

02 **Version parity creates synchronization pressure.**

03 **Validation oracle shapes release strategy.**

04 **Failure cost determines gate rigor.**

SCOPE NOTE

Process models shown are engineering syntheses based on cited public practices. The framework does not claim proprietary internal workflow details.

SOURCE BASIS & CITATION INTEGRITY

[Ref 1] Sommerville, I. Software Engineering, 10th ed. Pearson, 2015. [Ref 5] F5 NGINX Developer Blog, “Adopting Microservices at Netflix: Lessons for Architectural Design” (2015). [Ref 6] Riot Games Engineering, “The Riot Games API: Deep Dive” and “Keeping Legacy Software Alive: A Case Study” (2016–2021).

Original visuals by Truong Phat Tu • Analytical placements in Figure 2A are illustrative, not quantitative measurements.